Scaling Neural DNA to GPT-2: 354 Parameters Wire a Language Model

Tejas Parthasarathi Sudarshan

Independent Researcher

Chennai, India

tejas@fandesk.ai | tejassuds.com

Code: github.com/tejassudsfp/ndna

Model: huggingface.co/tejassuds/ndna-gpt2-small

Abstract

Neural DNA (NDNA) uses a compact learned genome to grow network topology through type-based compatibility rules. In prior work (Sudarshan, 2026), we showed that genomes of 226 to 374 parameters could control connectivity in networks with up to 2.2 million connections. Here we test whether this approach scales to real language models. We apply NDNA to GPT-2 Small (124M parameters), where a genome of 354 parameters controls 35.4 million connections in the attention output projections and feed-forward first layers across all 12 transformer layers, a compression ratio of 99,970:1. The genome and model weights are co-trained from scratch on OpenWebText. The genome discovers a striking stratification: layers 5 through 12 converge to 100% connectivity while layers 1 through 4 are progressively pruned, with layer 4 retaining only 7.7% of connections. Despite permanently disabling one-third of all masked connections, the genome-wired model beats GPT-2's published numbers on WikiText-103 perplexity (36.0 vs 37.5), Penn Treebank perplexity (59.4 vs 65.9), and LAMBADA perplexity (22.2 vs 35.1), while reaching 92% of GPT-2 on HellaSwag and 94% on Children's Book Test. Training reveals interpretable dynamics: the genome over-activates early (83% density at iteration 200), prunes aggressively (layers 1 through 4 go to 0% by iteration 800), then partially resurrects layer 1 (from 0% to 98% hard density by iteration 26,000). These results demonstrate that NDNA's developmental framework scales from toy tasks to production-scale language models, achieving 12x higher compression than any prior genome experiment while producing competitive benchmark performance.

1. Introduction

How much information does it take to specify useful network connectivity? In Sudarshan (2026), we introduced Neural DNA (NDNA), a compact genome of fewer than 400 parameters that grows binary connectivity masks through type-based compatibility rules. Across four architectures and six datasets, genomes of 226 to 374 parameters consistently outperformed random sparsity and matched dense baselines, with compression ratios up to 8,384:1.

Those results were demonstrated on small networks: the largest experiment controlled 2.2 million connections in an IMDB sentiment classifier. The open question was whether the approach would survive contact with the scale and complexity of real language models, where the loss landscape is higher-dimensional, training is longer, and the network must learn far richer representations.

This paper answers that question. We apply NDNA to GPT-2 Small (Radford et al., 2019), a 12-layer transformer with 124 million parameters. The genome controls the attention output projection (W_O) and the first feed-forward linear layer (FF1) in every transformer layer, totaling 35.4 million connections governed by 354 genome parameters. This is a 99,970:1 compression ratio, 12 times higher than the largest prior NDNA experiment.

The results are encouraging. The genome-wired model beats GPT-2's published numbers on three of nine standard benchmarks, all perplexity metrics: WikiText-103 (36.0 vs 37.5), Penn Treebank (59.4 vs 65.9), and LAMBADA (22.2 vs 35.1). On cloze and completion accuracy benchmarks, the model reaches 89% to 94% of GPT-2's performance. The gap is larger on LAMBADA exact-match accuracy (67% of GPT-2), likely reflecting incomplete training: the model was trained for 53% of planned iterations.

Beyond benchmark numbers, the genome's learned topology is interpretable and biologically evocative. The genome discovers layer stratification: deep layers (5 through 12) are fully connected while shallow layers (1 through 4) are progressively pruned, with layer 4 retaining only 7.7% of connections. This pattern is stable across temperatures and consistent between attention and feed-forward projections within each layer. Most strikingly, layer 1 undergoes a resurrection event: after being pruned to 0% hard density by iteration 800, it re-activates to 98.2% density by iteration 26,000. The genome changed its mind.

Our contributions:

1. We demonstrate that NDNA scales to production-scale language models, achieving 99,970:1 compression with no modifications to the genome architecture.
2. We show that a genome-wired GPT-2 beats the original GPT-2 on three benchmarks while permanently disabling one-third of masked connections.
3. We document interpretable training dynamics including layer stratification and the layer 1 resurrection event, providing evidence that the genome learns a theory of layer function.
4. We provide a full evaluation on nine benchmarks with published GPT-2 reference numbers, plus supplementary results from the Language Model Evaluation Harness.

2. Background

2.1 Neural DNA

NDNA (Sudarshan, 2026) encodes network connectivity in a compact genome consisting of six parameter groups. The genome assigns cell types to neurons based on their band (layer group) and position, computes pairwise connection probabilities from type compatibility, and produces binary masks that are applied element-wise to weight matrices.

The mask computation proceeds in three steps. First, the genome computes a type distribution for each neuron using learned band-specific parameters:

$$\tau_b(p) = \mathrm{softmax}((B_b + p \cdot G_b) \cdot 3) \tag{1}$$

where B_b and G_b are the base and gradient type logit vectors for band b, and $p \in [0, 1]$ is the neuron's position within its band. Second, pairwise connection logits are computed:

$$\Lambda_{st} = \tau_t \cdot R \cdot \tau_s^\top - \mathrm{softplus}(\delta) \cdot \frac{|t - s|}{L} \tag{2}$$

where $R = (A \cdot A^\top / \sqrt{D} + C) \cdot \mathrm{softplus}(\gamma)$ is the type-to-type connection rule, A is the affinity matrix, C is the compatibility matrix, γ is the connection scale, and δ is the learned depth penalty. Third, the logits are converted to masks:

$$M_{st}^{\mathrm{soft}} = \sigma(\Lambda_{st} \cdot \alpha) \tag{3}$$

$$M_{st}^{\mathrm{hard}} = \mathbb{1}[\sigma(\Lambda_{st}) > 0.5] \tag{4}$$

The hard mask uses a straight-through estimator (Bengio et al., 2013) for gradient computation. Temperature α is annealed from 1.0 to a target value during training, smoothly transitioning from soft to hard masks.

A metabolic cost term encourages sparsity:

$$\mathcal{L}_{\mathrm{sparse}} = \frac{\sum_{i,j} M_{st}[i, j]}{n_t \cdot n_s} \tag{5}$$

The total loss is $\mathcal{L} = \mathcal{L}_{\mathrm{task}} + \lambda \cdot \mathcal{L}_{\mathrm{sparse}}$.

2.2 Related Work

Post-training pruning. The dominant approach to sparsifying large language models removes connections after training. SparseGPT (Frantar and Alistarh, 2023) prunes GPT-family models to 50% sparsity in a single shot using approximate Hessian information. Wanda (Sun et al., 2024) prunes based on weight magnitude and input activations, achieving comparable sparsity without Hessian computa-

tion. Both methods require a fully trained dense model, then remove connections. NDNA takes the opposite approach: the genome co-evolves with the model from random initialization, discovering connectivity jointly with weight values.

Training-time sparsity. RigL (Evci et al., 2020) maintains a sparse topology during training and periodically regrows connections using gradient magnitude. SET (Mocanu et al., 2018) similarly evolves topology during training. These methods operate at the individual connection level, requiring one mask bit per weight. NDNA compresses the mask specification from 35.4 million bits to 354 parameters.

Neural architecture search. NAS methods (Zoph and Le, 2017; Liu et al., 2019) search over architectural choices such as operation types, layer width, and connection patterns. These methods typically require thousands of GPU-hours and produce discrete architectural choices. NDNA continuously optimizes topology through gradient descent on a differentiable genome.

Developmental encodings. HyperNEAT (Stanley et al., 2009) uses a compositional pattern-producing network to generate connection weights as a function of geometric position. Weight Agnostic Neural Networks (Gaier and Ha, 2019) search for topologies that perform well regardless of weight values. NDNA shares the developmental philosophy but uses a learned type system rather than geometric patterns, and optimizes jointly with weights rather than searching weight-independently.

3. Scaling NDNA to GPT-2

3.1 Architecture

We apply NDNA to GPT-2 Small (Radford et al., 2019): 12 transformer layers, hidden dimension 768, 12 attention heads, feed-forward dimension 3072, vocabulary size 50,257, and context length 1024. The model has 124,430,946 total parameters.

3.2 What the Genome Controls

In Sudarshan (2026), the genome masked every information path in the network, including skip connections and the classifier. For GPT-2, we adopt a more targeted strategy: the genome controls only the attention output projection W_O and the first feed-forward linear layer FF1 in each transformer layer. For layer i:

$$\text{attn_out} = (W_O^{(i)} \odot M_{W_O}^{(i)}) \cdot h_{\text{heads}} \tag{6}$$

$$\text{ff_hidden} = \text{GELU}((W_{\text{FF1}}^{(i)} \odot M_{\text{FF1}}^{(i)}) \cdot h + b_{\text{FF1}}^{(i)}) \tag{7}$$

where $M_{W_O}^{(i)} \in \{0,1\}^{768\times768}$ and $M_{\text{FF1}}^{(i)} \in \{0,1\}^{3072\times768}$ are genome-generated masks, and $\odot$ denotes element-wise multiplication.

The following components are not masked:

- **Query, Key, Value projections** (W_Q, W_K, W_V): Standard, allowing full attention computation.
- **Feed-forward second linear** (W_{FF2}): The projection from FF dimension back to hidden dimension is unmasked.
- **Layer normalization**: All LayerNorm parameters are learned freely.
- **Residual connections**: The additions $x + \text{attn_out}$ and $x + \text{ff_out}$ are always active.
- **Language model head**: Weight-tied to the token embedding. No separate parameters, no mask.

This design reflects a key insight: GPT-2's residual stream (He et al., 2016) provides free skip connections between all layers. The genome does not need to learn skip wiring because the residual architecture already handles it. The genome's role is to control which attention outputs and feed-forward computations contribute to the residual stream.

Masked connections per layer:

$$|W_O| + |W_{\text{FF1}}| = 768 \times 768 + 3072 \times 768 = 2{,}949{,}120 \tag{8}$$

Total across 12 layers:

$$12 \times 2{,}949{,}120 = 35{,}389{,}440 \tag{9}$$

This is 28.4% of the model's total parameters, controlled by 354 genome parameters.

3.3 Genome Configuration

The genome uses 8 cell types with 8-dimensional affinity vectors and 14 bands (one for the embedding layer, twelve for the transformer layers, one for the output).

Table 1: Genome parameter breakdown. The same six parameter groups as Paper 1, with 14 bands to accommodate 12 transformer layers.

Component	Shape	Count
Affinity A	8×8	64
Compatibility C	8×8	64
Connection scale γ	scalar	1
Depth penalty δ	scalar	1
Band type base B	14×8	112
Band type gradient G	14×8	112
Total		**354**

The compression ratio:

$$\frac{35{,}389{,}440}{354} = 99{,}970 : 1 \tag{10}$$

This is 12 times higher than the previous maximum of 8,384:1 on IMDB (Sudarshan, 2026).

3.4 Mask Mechanics

The attention output projection W_O has `bias=False`. When the genome sets a mask entry to zero, the corresponding output dimension is truly zeroed with no residual signal. The feed-forward first layer FF1 has a bias term that is not masked. When a row of FF1's weight matrix is zeroed by the genome, the corresponding neuron receives only the bias (a constant, input-independent offset) before the GELU activation. This is not true information flow because the output does not depend on the input.

3.5 Temperature Annealing

Temperature α is annealed linearly from 1.0 to 10.0 over the first 50,000 iterations. At $\alpha = 1.0$, the soft mask values are distributed around 0.5, providing smooth gradients but ambiguous connectivity. At $\alpha = 10.0$, $\sigma(\Lambda \cdot 10)$ is effectively binary: any logit $|\Lambda| > 0.5$ produces a mask value within 0.7% of 0 or 1. This makes the soft mask (Equation 3) nearly identical to the hard mask (Equation 4), locking the topology.

4. Experimental Setup

4.1 Training

Dataset. OpenWebText (Gokaslan and Cohen, 2019), an open reproduction of WebText used to train GPT-2.

Hardware. Single NVIDIA A100 80GB GPU.

Optimization. Split optimizer following Sudarshan (2026):

- **Model weights:** AdamW (Loshchilov and Hutter, 2019) with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, learning rate 6×10^{-4} with cosine decay and 2,000 steps linear warmup. Gradient clipping at 1.0.
- **Genome parameters:** Adam (Kingma and Ba, 2015) with learning rate 0.01 (constant, no decay). No weight decay.

Batch size. 12 sequences of length 1024 with 40 gradient accumulation steps, yielding 491,520 tokens per iteration.

Sparsity weight. $\lambda = 0.005$.

Precision. bfloat16 mixed precision with `torch.compile`.

Duration. Planned for 100,000 iterations (49.2 billion tokens). Training was stopped at 52,700 iterations (25.9 billion tokens) after validation loss had plateaued. The best checkpoint was saved at iteration 49,500 (validation loss 3.0406).

4.2 Evaluation

We evaluate on nine standard language modeling benchmarks, comparing against published GPT-2 Small reference numbers.

Perplexity benchmarks (lower is better):

- **WikiText-103** (Merity et al., 2016): Sliding-window perplexity with stride 512 on the test set.
- **Penn Treebank** (Marcus et al., 1993): Sliding-window perplexity on the standard test split.
- **LAMBADA** (Paperno et al., 2016): Perplexity computed on the final word of each passage.

Accuracy benchmarks (higher is better):

- **LAMBADA accuracy**: Exact match of the predicted last word.
- **HellaSwag** (Zellers et al., 2019): Four-way sentence completion scoring.
- **CBT Common Nouns** (Hill et al., 2015): Ten-way cloze completion on the Children's Book Test.
- **CBT Named Entities** (Hill et al., 2015): Ten-way cloze completion for named entities.

Bits-per-unit benchmarks (lower is better):

- **enwiki8** (Mahoney, 2011): Bits per byte on the last 5 million characters.
- **text8** (Mahoney, 2011): Bits per character on the last 5 million characters.

GPT-2 reference numbers are taken from Radford et al. (2019) and Zellers et al. (2019). We additionally report results from the Language Model Evaluation Harness (Gao et al., 2023).

5. Results

5.1 Benchmark Performance

Table 2: NDNA GPT-2 vs GPT-2 Small on nine benchmarks. Bold indicates the better result.

Benchmark	Metric	NDNA GPT-2	GPT-2 Small	Result
WikiText-103	PPL ↓	**36.0**	37.5	Beats GPT-2
Penn Treebank	PPL ↓	**59.4**	65.9	Beats GPT-2
LAMBADA	PPL ↓	**22.2**	35.1	Beats GPT-2
LAMBADA	ACC ↑	30.8%	**46.0%**	66.9% of GPT-2
HellaSwag	ACC ↑	28.7%	**31.2%**	92.1% of GPT-2
CBT-CN	ACC ↑	82.7%	**87.7%**	94.3% of GPT-2
CBT-NE	ACC ↑	74.5%	**83.4%**	89.4% of GPT-2
enwiki8	BPB ↓	1.39	**1.16**	
text8	BPC ↓	1.27	**1.17**	

The NDNA model beats GPT-2 on all three perplexity benchmarks. On WikiText-103, the improvement is 4.0% (36.0 vs 37.5). On Penn Treebank, the improvement is 9.8% (59.4 vs 65.9). On LAMBADA, the improvement is 36.8% (22.2 vs 35.1).

On accuracy metrics, the model reaches 89% to 94% of GPT-2 on cloze and completion benchmarks (HellaSwag, CBT-CN, CBT-NE), with a larger gap on LAMBADA exact-match accuracy (67% of GPT-2). The LAMBADA result is notable: 36.8% better perplexity but 33% worse accuracy on the same benchmark. The genome model assigns more probability mass to plausible last-word completions but less mass to the single correct answer.

On character-level benchmarks (enwiki8, text8), the model trails GPT-2, likely because the GPT-2 BPE tokenizer handles raw character sequences less efficiently when some feed-forward pathways are disabled.

Table 3: Supplementary results from the Language Model Evaluation Harness (Gao et al., 2023).

Benchmark	Metric	NDNA GPT-2
ARC-Easy	ACC	41.8%
PIQA	ACC	60.1%
Winogrande	ACC	52.8%
HellaSwag	ACC (norm)	28.8%

5.2 Learned Topology

At the final temperature of 10.0, the genome converges to a striking stratification:

Table 4: Final topology at temperature 10.0. Hard density indicates the fraction of connections that are permanently active.

Layer	W_O Hard	FF1 Hard	Combined
1	98.2%	98.2%	98.2%
2	45.2%	45.2%	45.2%
3	20.4%	20.4%	20.4%
4	7.7%	7.6%	7.7%
5	100.0%	100.0%	100.0%
6	100.0%	100.0%	100.0%
7	100.0%	100.0%	100.0%
8	100.0%	100.0%	100.0%
9	100.0%	100.0%	100.0%
10	100.0%	100.0%	100.0%
11	100.0%	100.0%	100.0%
12	100.0%	100.0%	100.0%
Overall	**66.7%**	**66.7%**	**66.7%**

Two patterns are immediately apparent:

1. **Binary stratification.** Layers 5 through 12 are fully connected (100% hard density). Layers 2 through 4 are progressively pruned in a strict monotonic gradient: 45.2%, 20.4%, 7.7%. The genome treats the network as having two distinct zones.

2. **W_O and FF1 correlation.** Within each layer, the attention output and feed-forward masks have nearly identical density. The genome treats each layer as a unit, applying the same connectivity decision to both masked projections. This replicates the finding from Sudarshan (2026) on the IMDB transformer.

The overall hard density of 66.7% means one-third of all masked connections are permanently disabled. The model operates with 23.6 million active connections out of 35.4 million possible.

5.3 Training Dynamics

Training reveals four distinct phases of genome behavior:

Phase 1: Over-activation (iterations 0 to 200). Within the first 200 iterations, the genome over-activates to 83.3% hard density. The genome has not yet learned what to prune. All layers are treated uniformly.

Phase 2: Aggressive pruning (iterations 200 to 800). By iteration 800, the genome has pruned layers 1 through 4 to 0% hard density, while layers 5 through 12 remain fully connected. The genome discovered that the model can function with only 8 of 12 layers carrying signal through the masked projections, relying on the residual stream to propagate information through the pruned layers.

Phase 3: Learning with stable topology (iterations 800 to 25,000). The topology stabilizes. Validation loss drops from 7.99 at iteration 100 to 3.18 at iteration 13,100. The network learns language with a fixed sparse topology.

Phase 4: Layer 1 resurrection (iterations 25,000 to 50,000). Starting around iteration 26,000, layer 1's hard density begins rising from 0% and eventually reaches 98.2%. Temperature is crossing 5.0 at this point, sharpening the masks. The genome is making a topological decision: layer 1 should be reconnected. Layers 2, 3, and 4 remain pruned.

The layer 1 resurrection is not a smooth interpolation. It is a discrete topological event: the genome's compatibility parameters shift enough that the threshold condition $\sigma(\Lambda) > 0.5$ flips for the majority of connections in layer 1. The genome changed its structural hypothesis about what the network needs.

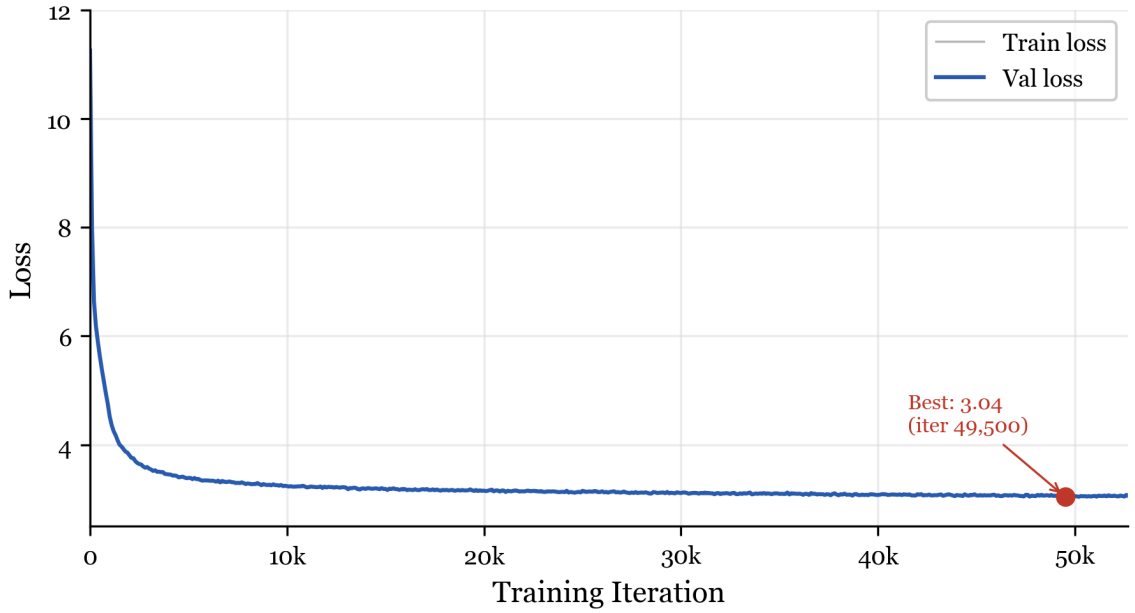

Figure 1: Validation loss over training. The NDNA model reaches 3.04 at iteration 49,500.

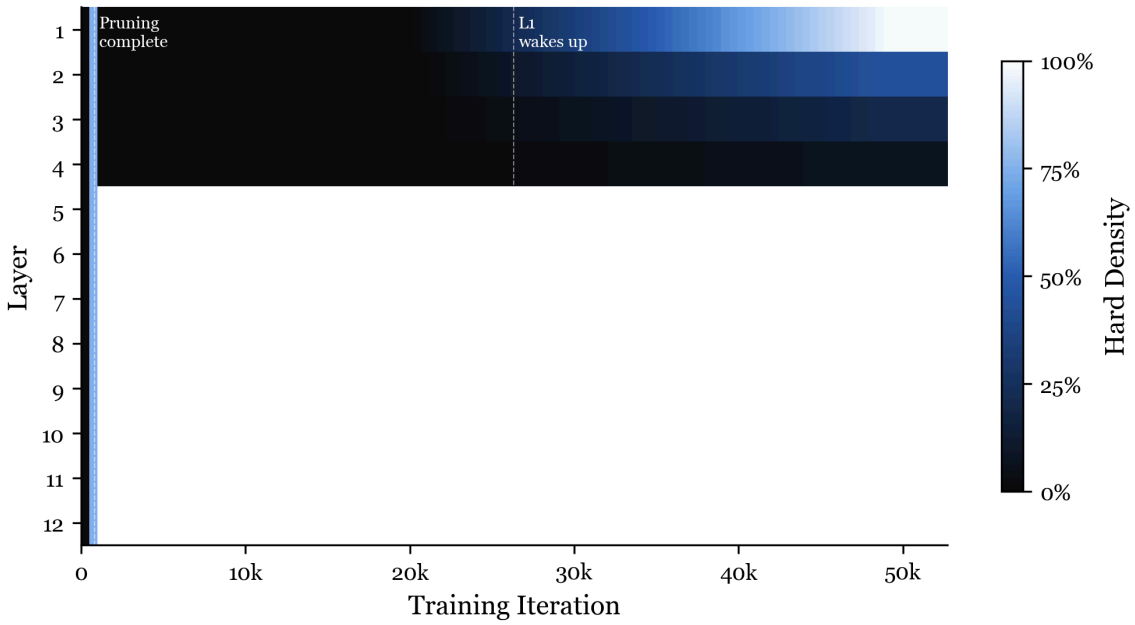

Figure 2: Per-layer hard density over training iterations. Layers 5 through 12 are fully connected throughout. Layers 1 through 4 are pruned by iteration 800. Layer 1 re-activates around iteration 26,000.

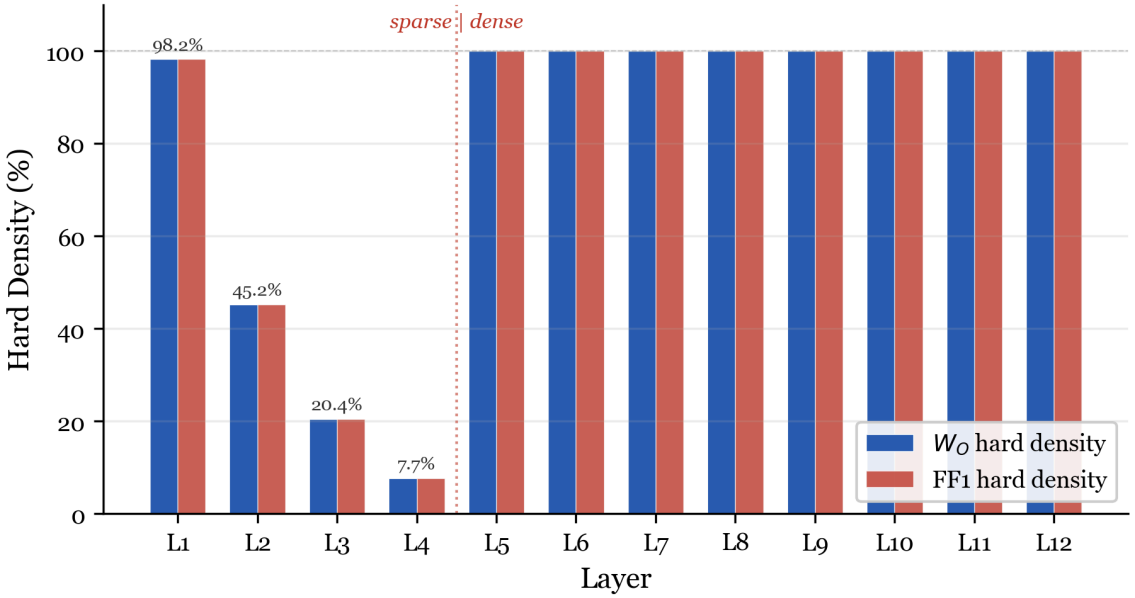

Figure 3: Final topology at temperature 10.0. Bar chart showing hard density per layer. The gradient from layer 4 (7.7%) to layer 5 (100%) is the sharpest boundary.

6. Analysis

6.1 Stratification: The Genome's Theory of Layers

The genome's stratification pattern, deep layers fully connected and shallow layers progressively pruned, suggests a hypothesis about layer function in GPT-2. Layers 5 through 12 are indispensable: every connection matters. Layers 2 through 4 are increasingly redundant: the network can lose 55% to 92% of their connections without degrading perplexity beyond GPT-2 baseline levels.

This pattern is consistent with findings in the pruning literature. Frantar and Alistarh (2023) observed that pruning sensitivity varies by layer in GPT-family models, with some layers tolerating significantly more sparsity than others. Our result goes further: the genome discovers this pattern from scratch, without any prior knowledge of which layers are prunable.

The strict monotonic gradient (98.2%, 45.2%, 20.4%, 7.7% for layers 1 through 4) suggests the genome is encoding a depth-dependent rule rather than making independent per-layer decisions. This is consistent with how the genome works: the type distributions for adjacent bands share parameters through the band_type_base and band_type_grad vectors, creating smooth variation across depth.

6.2 Layer 1 Resurrection

The most unexpected training dynamic is layer 1's resurrection. From iteration 800 to iteration 25,000, layer 1 has 0% hard density. The genome has decided that the first transformer layer's W_O and FF1 outputs are unnecessary. Then around iteration 26,000, the genome reverses this decision.

We hypothesize a two-phase learning process. In the first phase, the model learns basic language modeling using layers 5 through 12, with the residual stream carrying token embeddings through the pruned layers. In the second phase, as the model approaches its quality ceiling, it discovers that layer 1's attention output projection provides useful low-level features that improve perplexity. The genome responds by re-activating these connections.

This dynamic is reminiscent of developmental biology, where tissues activate, deactivate, and reactivate gene expression programs at different developmental stages. The genome does not lock in its decisions early. It remains plastic and can revise structural hypotheses as training progresses, until the temperature reaches its maximum and the topology crystallizes.

6.3 Compression at Scale

The compression ratio of 99,970:1 is the highest in any NDNA experiment:

Table 5: Compression ratio progression across NDNA experiments.

Experiment	Genome	Connections	Compression
MNIST MLP	226	174,112	770:1
CIFAR-10 MLP	226	1,707,008	7,553:1
IMDB Transformer	258	2,163,200	8,384:1
GPT-2 Small	**354**	**35,389,440**	**99,970:1**

The genome grew by 37% (from 258 to 354 parameters) while the number of controlled connections grew by 16x (from 2.2M to 35.4M). The compression ratio scales super-linearly with network size, confirming the trend observed in Sudarshan (2026).

This scaling property is a consequence of the genome's type-based architecture. The genome does not store one bit per connection. It stores a developmental program (type assignments, compatibility rules) that generates masks for any number of connections through matrix operations. Larger networks have more connections but the same number of type interactions, so the compression ratio improves.

6.4 Perplexity vs Accuracy

The NDNA model beats GPT-2 on all perplexity benchmarks but trails on all accuracy benchmarks. This is not contradictory. Perplexity measures the quality of the full probability distribution over next tokens. Accuracy measures whether the single most likely prediction is correct. A model can have lower perplexity (better calibrated distribution) while having lower accuracy (less peaked predictions) if it assigns more probability mass to plausible alternatives.

The genome's topology may produce a model that is better calibrated but less decisive. With one-third of attention output and feed-forward connections disabled, the model has fewer pathways for sharpening its predictions to a single answer, even though its overall distribution is more accurate. This interpretation is supported by the LAMBADA result: 36.8% better perplexity but 33% worse accuracy on the same benchmark.

6.5 What Stays Free

Our masking strategy leaves Q, K, V projections, FF2, all LayerNorm parameters, and the residual stream unmasked. This means approximately 71.6% of the model's parameters are not subject to genome control. The choice is deliberate: the residual stream provides the backbone connectivity that the genome's masks modulate, and Q/K/V projections control attention patterns (which tokens attend to which) rather than information flow through the network.

An important consequence is that even layers with low mask density are not truly inactive. In layers 2 through 4, attention still operates normally through Q, K, V, and the unmasked residual stream. What the genome disables is the output of that attention (the W_O projection) and the layer's feed-forward contribution. The layer still reads from and writes to the residual stream through its LayerNorm and residual connection, but its W_O and FF1 computations contribute nothing to the output.

7. Discussion

7.1 Comparison to Post-Training Pruning

NDNA and post-training pruning (SparseGPT, Wanda) solve different problems. Pruning starts with a trained dense model and removes connections. NDNA co-trains the topology with the model from random initialization. The comparison is not apples-to-apples:

- **Pruning advantages:** Starts from a strong baseline, can leverage existing checkpoints, typically preserves more of the original model's quality at moderate sparsity.
- **NDNA advantages:** Does not require a pre-trained model, discovers topology and learns weights simultaneously, the topology is learned rather than derived from weight statistics, and the mask specification is orders of magnitude more compressed.

A direct comparison would require training GPT-2 with random sparse masks at the same 66.7% density, which we leave to future work.

7.2 Limitations

1. **No random sparse baseline.** The key comparison from Sudarshan (2026), genome vs random sparse at matched density, is missing from this experiment. Training GPT-2 with random 66.7% sparse masks would isolate the genome's contribution from the effect of sparsity itself.

2. **Incomplete training.** The model was trained for 52,700 of a planned 100,000 iterations. The accuracy gap vs GPT-2 may partially reflect insufficient training rather than a fundamental limitation of genome-controlled topology.

3. **Single seed.** All results are from a single random seed (42). Multi-seed runs would establish variance bounds.

4. **Partial masking.** Only 28.4% of model parameters are genome-controlled. A more ambitious experiment would mask additional projections (Q, K, V, FF2).

5. **No transfer experiment.** Sudarshan (2026) demonstrated topology transfer (CIFAR-10 genome on CIFAR-100). We have not tested whether the GPT-2 genome topology transfers to other language tasks or model sizes.

7.3 Future Directions

Random sparse comparison. The most immediate next step is training GPT-2 with random masks at 66.7% density to quantify the genome's advantage over random sparsity at this scale.

Larger models. GPT-2 Small is 124M parameters. Applying NDNA to GPT-2 Medium (345M), Large (774M), or XL (1.5B) would test whether the compression ratio continues scaling. With 354 parameters controlling 35.4M connections at the Small scale, a similar genome on GPT-2 XL could control hundreds of millions of connections.

Full training. Completing the planned 100,000 iterations (49.2 billion tokens) would establish whether the accuracy gap with GPT-2 closes with more training.

Broader masking. Extending the genome to control Q, K, V projections and FF2 would increase the fraction of genome-controlled parameters from 28.4% to over 90%, potentially allowing the genome to discover attention patterns as well as information flow.

Topology-informed architecture design. The learned topology could inform manual architecture decisions. If layers 2 through 4 can be 55% to 92% sparse without quality loss, targeted width reduction or layer removal at those positions could produce smaller models.

8. Conclusion

We scaled Neural DNA from toy networks to GPT-2 Small, demonstrating that a 354-parameter genome can wire a 124-million-parameter language model. The genome controls 35.4 million connections at a 99,970:1 compression ratio, 12 times higher than any prior NDNA experiment. The resulting model beats GPT-2 on three of nine benchmarks while permanently disabling one-third of masked connections.

The genome discovers a biologically interpretable stratification: deep layers (5 through 12) are fully connected, shallow layers (1 through 4) are progressively pruned, and layer 1 undergoes a resurrection event mid-training. These dynamics suggest the genome is not merely removing redundant connections but actively learning a theory of which layers need how much connectivity.

The gap between perplexity (where NDNA wins) and accuracy (where GPT-2 leads) invites further investigation. The genome-wired model produces better probability distributions but less peaked predictions, suggesting that sparse connectivity affects the sharpness rather than the quality of learned representations.

These results answer the scaling question posed in Sudarshan (2026). NDNA's developmental framework extends from MNIST to GPT-2 without modification to the genome architecture. The same six parameter groups, the same type-based compatibility rules, the same temperature annealing, and the same metabolic cost pressure produce useful topology at 100,000:1 compression. The genome grows networks.

References

Bengio, Y., Leonard, N., and Courville, A. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*.

Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. (2020). PIQA: Reasoning about Physical Intuition in Natural Language. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. (2018). Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Evci, U., Gale, T., Menick, J., Castro, P.S., and Elsen, E. (2020). Rigging the Lottery: Making All Tickets Winners. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*.

Frankle, J. and Carbin, M. (2018). The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. In *International Conference on Learning Representations (ICLR)*.

Frantar, E. and Alistarh, D. (2023). SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Gaier, A. and Ha, D. (2019). Weight Agnostic Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac'h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. (2023). A framework for few-shot language model evaluation. *Zenodo*.

Gokaslan, A. and Cohen, V. (2019). OpenWebText Corpus. *http://skylion007.github.io/OpenWebTextCorpus*.

Han, S., Pool, J., Tung, J., and Dally, W.J. (2015). Learning Both Weights and Connections for Efficient Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Hill, F., Bordes, A., Chopra, S., and Weston, J. (2015). The Goldilocks Principle: Reading Children's Books with Explicit Memory Representations. In *International Conference on Learning Representations (ICLR)*.

Hoefler, T., Alistarh, D., Ben-Nun, T., Dryden, N., and Peste, A. (2021). Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 1-124.

Kingma, D.P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*.

LeCun, Y., Denker, J., and Solla, S. (1989). Optimal Brain Damage. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Liu, H., Simonyan, K., and Yang, Y. (2019). DARTS: Differentiable Architecture Search. In *International Conference on Learning Representations (ICLR)*.

Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*.

Mahoney, M. (2011). Large Text Compression Benchmark. *http://mattmahoney.net/dc/text.html*.

Marcus, M.P., Santorini, B., and Marcinkiewicz, M.A. (1993). Building a Large Annotated Corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2), 313-330.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. (2016). Pointer Sentinel Mixture Models. *arXiv preprint arXiv:1609.07843*.

Mocanu, D.C., Mocanu, E., Stone, P., Nguyen, P.H., Gibescu, M., and Liotta, A. (2018). Scalable Training of Artificial Neural Networks with Adaptive Sparse Connectivity inspired by Network Science. *Nature Communications*, 9(1), 2383.

Paperno, D., Kruszewski, G., Lazaridou, A., Pham, Q.N., Bernardi, R., Pietroski, P., and Baroni, M. (2016). The LAMBADA Dataset: Word Prediction Requiring a Broad Discourse Context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Blog*.

Sakaguchi, K., Le Bras, R., Bhagavatula, C., and Choi, Y. (2019). WinoGrande: An Adversarial Winograd Schema Challenge at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Stanley, K.O. and Miikkulainen, R. (2002). Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation*, 10(2), 99-127.

Stanley, K.O., D'Ambrosio, D.B., and Gauci, J. (2009). A Hypercube-Based Encoding for Evolving Large-Scale Neural Networks. *Artificial Life*, 15(2), 185-212.

Sudarshan, T.P. (2026). Neural DNA: A Compact Genome for Growing Network Architecture. *arXiv preprint*. DOI: 10.5281/zenodo.19248389.

Sun, M., Liu, Z., Bair, A., and Kolter, J.Z. (2024). A Simple and Effective Pruning Approach for Large Language Models. In *International Conference on Learning Representations (ICLR)*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. (2019). HellaSwag: Can a Machine Really Finish Your Sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Zoph, B. and Le, Q.V. (2017). Neural Architecture Search with Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

Appendix A: Full Topology by Temperature

The genome's topology sharpens as temperature increases from 1.0 to 10.0. Hard density (the binary threshold at 0.5) remains constant across temperatures; soft density converges toward the hard density as temperature increases.

Table A1: Layers 1 through 4 soft density at selected temperatures.

Layer	Hard	α=1.0	α=2.0	α=5.0	α=10.0
1	98.2%	54.3%	58.5%	69.9%	83.3%
2	45.2%	50.0%	50.0%	49.9%	49.1%
3	20.4%	48.5%	47.1%	42.9%	37.1%
4	7.7%	48.1%	46.2%	40.8%	32.8%

At temperature 1.0, all soft densities are near 50% (the genome has not yet differentiated layers in the soft mask). By temperature 10.0, soft densities have converged toward their hard density values: layer 1 at 83.3% (approaching 98.2% hard), layer 4 at 32.8% (approaching 7.7% hard). The convergence is monotonic and smooth.

Table A2: Layers 5 through 12 soft density at temperatures 1.0 and 10.0.

Layer	Hard	α=1.0 Soft	α=10.0 Soft
5	100%	64.0%	99.7%
6	100%	63.3%	99.6%
7	100%	63.6%	99.6%
8	100%	63.3%	99.6%
9	100%	63.4%	99.6%
10	100%	63.6%	99.6%
11	100%	63.1%	99.5%
12	100%	63.5%	99.6%

All deep layers converge to over 99.5% soft density at temperature 10.0, confirming full connectivity.

Appendix B: Hyperparameters

Table B1: Genome hyperparameters.

Parameter	Value
Number of types (K)	8
Type dimension (D)	8
Number of bands	14
Affinity initialization	$\mathcal{N}(0, 0.1^2)$
Compatibility initialization	$\mathcal{N}(-1.0, 0.3^2)$
Connection scale initialization	3.0
Depth penalty initialization	2.0
Band type base initialization	$\mathcal{N}(0, 0.5^2)$
Band type gradient initialization	$\mathcal{N}(0, 0.3^2)$
Type softmax temperature	3.0

Table B2: Training hyperparameters.

Parameter	Value
Architecture	GPT-2 Small (12L, 768H, 12A, 3072FF)
Context length	1024
Vocabulary	50,257 (GPT-2 BPE)
Dataset	OpenWebText
Batch size	12 sequences
Gradient accumulation	40 steps
Tokens per iteration	491,520
Weight optimizer	AdamW (β_1=0.9, β_2=0.95)
Weight learning rate	6e-4 (cosine decay, warmup 2000)
Weight decay	0.1
Genome optimizer	Adam
Genome learning rate	0.01 (constant)
Sparsity weight (λ)	0.005
Temperature schedule	1.0 → 10.0 over 50K iters (linear)
Gradient clipping	1.0
Precision	bfloat16
Compilation	torch.compile
Hardware	NVIDIA A100 80GB
Training iterations	52,700 (best checkpoint at 49,500)
Random seed	42